

Sprawozdanie numer 4

Bartosz Bodulski, grupa 1, tematy 12 i 13.

Temat 12

Cel zajęć: Zapoznanie się z github Actions oraz dyrektywą "shift-left".

Na początku zgodnie z poleceniem, forkuje i klonuje repozytorium mojego wcześniej wybranego oprogramowania - jest to neovim, tworzę nową gałąź lokalną ino_dev, usuwam wszystkie workflowy dla github actions ustawione przez autorów oprogramowania.

```
● bartosz123@ansible-master:~/MDO2026s_ITE$ 1
ITE/ README.md READMEs/
● bartosz123@ansible-master:~/MDO2026s_ITE$ cd ITE/GCL1/BB419678/lab12/
● bartosz123@ansible-master:~/MDO2026s_ITE/ITE/GCL1/BB419678/lab12$ git clone https://github.com/Bogdomeble/neovim.git
Cloning into 'neovim'...
remote: Enumerating objects: 310439, done.
remote: Counting objects: 100% (110/110), done.
remote: Compressing objects: 100% (66/66), done.
remote: Total 310439 (delta 74), reused 44 (delta 44), pack-reused 310329 (from 2)
Receiving objects: 100% (310439/310439), 361.28 MiB | 5.86 MiB/s, done.
● bartosz123@ansible-master:~/MDO2026s_ITE/ITE/GCL1/BB419678/lab12$ cd neovim/
● bartosz123@ansible-master:~/MDO2026s_ITE/ITE/GCL1/BB419678/lab12/neovim$ git checkout -b ino_dev
Switched to a new branch 'ino_dev'
● bartosz123@ansible-master:~/MDO2026s_ITE/ITE/GCL1/BB419678/lab12/neovim$ git rm -rf .github/workflows/*
rm '.github/workflows/backport.yml'
rm '.github/workflows/build.yml'
rm '.github/workflows/build_dummy.yml'
rm '.github/workflows/codeql.yml'
rm '.github/workflows/coverity.yml'
rm '.github/workflows/docs.yml'
rm '.github/workflows/labeler_issue.yml'
rm '.github/workflows/labeler_pr.yml'
rm '.github/workflows/lintcommit.yml'
rm '.github/workflows/lintcommit_dummy.yml'
rm '.github/workflows/lintdocurls.yml'
rm '.github/workflows/news.yml'
rm '.github/workflows/notes.md'
rm '.github/workflows/optional.yml'
rm '.github/workflows/release.yml'
rm '.github/workflows/response.yml'
rm '.github/workflows/reviewers_add.yml'
rm '.github/workflows/reviewers_remove.yml'
rm '.github/workflows/test.yml'
rm '.github/workflows/test_windows.yml'
rm '.github/workflows/vim_patches.yml'
rm '.github/workflows/zizmor.yml'
● bartosz123@ansible-master:~/MDO2026s_ITE/ITE/GCL1/BB419678/lab12/neovim$ git commit -m "Usuniecie starych workflowow zgodnie z poleceniem"
[ino_dev 7288232660] Usuniecie starych workflowow zgodnie z poleceniem
22 files changed, 1610 deletions(-)
delete mode 100644 .github/workflows/backport.yml
delete mode 100644 .github/workflows/build.yml
delete mode 100644 .github/workflows/build_dummy.yml
delete mode 100644 .github/workflows/codeql.yml
delete mode 100644 .github/workflows/coverity.yml
delete mode 100644 .github/workflows/docs.yml
delete mode 100644 .github/workflows/labeler_issue.yml
delete mode 100644 .github/workflows/labeler_pr.yml
delete mode 100644 .github/workflows/lintcommit.yml
delete mode 100644 .github/workflows/lintcommit_dummy.yml
delete mode 100644 .github/workflows/lintdocurls.yml
delete mode 100644 .github/workflows/news.yml
delete mode 100644 .github/workflows/notes.md
delete mode 100644 .github/workflows/optional.yml
delete mode 100644 .github/workflows/release.yml
delete mode 100644 .github/workflows/response.yml
delete mode 100644 .github/workflows/reviewers_add.yml
delete mode 100644 .github/workflows/reviewers_remove.yml
delete mode 100644 .github/workflows/test.yml
delete mode 100644 .github/workflows/test_windows.yml
delete mode 100644 .github/workflows/vim_patches.yml
delete mode 100644 .github/workflows/zizmor.yml
```

Następnie dodaje swój plik dla Github Actions, w którym sprawdzam jakość kodu, gdyż neovim lubi się długo budować, szczególnie na 1 rdzeniu, który przydzielony jest w ramach planu darmowego.

```

bartosz123@ansible-master:~/MD02026s ITE$ cd ITE/GCL1/BB419678/lab12/neovim/.github/
actions/      ISSUE_TEMPLATE/ scripts/
bartosz123@ansible-master:~/MD02026s ITE$ cd ITE/GCL1/BB419678/lab12/neovim/.github/
actions/      ISSUE_TEMPLATE/ scripts/
bartosz123@ansible-master:~/MD02026s ITE$ cd ITE/GCL1/BB419678/lab12/neovim/.github
bartosz123@ansible-master:~/MD02026s ITE/ITE/GCL1/BB419678/lab12/neovim/.github$ mkdir workflows
bartosz123@ansible-master:~/MD02026s ITE/ITE/GCL1/BB419678/lab12/neovim/.github$ cd workflows/
bartosz123@ansible-master:~/MD02026s ITE/ITE/GCL1/BB419678/lab12/neovim/.github/workflows$ nano shift-left.yml
bartosz123@ansible-master:~/MD02026s ITE/ITE/GCL1/BB419678/lab12/neovim/.github/workflows$ git add .github/workflows/shift-left.yml
git commit -m "Dodanie własnej akcji Code Quality dla Neovima"
git push -u origin ino_dev
warning: could not open directory '.github/workflows/.github/workflows/': No such file or directory
fatal: pathspec '.github/workflows/shift-left.yml' did not match any files
On branch ino_dev
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  ./

nothing added to commit but untracked files present (use "git add" to track)
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 6 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 317 bytes | 317.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'ino_dev' on GitHub by visiting:
remote:   https://github.com/Bogdomeble/neovim/pull/new/ino_dev
remote:
To https://github.com/Bogdomeble/neovim.git
 * [new branch]      ino_dev -> ino_dev
branch 'ino_dev' set up to track 'origin/ino_dev'.
bartosz123@ansible-master:~/MD02026s ITE/ITE/GCL1/BB419678/lab12/neovim/.github/workflows$ █

```

Plik konfiguracyjny reaguje na pushe na gałęzi `ino_dev` dla systemu ubuntu 24.04. Tutaj mamy podobny workflow do naszego wcześniejszego Jenkinsfile, ale tutaj jest mamy uproszczenie całego pipeline'u. Najpierw pobieramy narzędzie shellcheck do sprawdzania skryptów powłoki bash w projekcie, po czym sprawdzamy je i na koniec tworzymy prosty raport w postaci artefaktu, pakując go w tarball'a o nazwie `neovim-raport-artefakt`.

```

name: Shift-left Neovim Pipeline

# Akcja reaguje tylko na kod wrzucony do gałęzi ino_dev
on:
  push:
    branches:
      - ino_dev

jobs:
  code-quality-and-artifact:
    runs-on: ubuntu-24.04

    steps:
      # Pobranie kodu repozytorium
      - name: Checkout repository
        uses: actions/checkout@v4

      # Analiza jakości kodu (Code Quality) - Zamiast długiego buildu
      - name: Sprawdzenie jakości kodu (Shellcheck)
        run: |
          echo "Zgodnie z poleceniem: Ponieważ pełny build/test Neovima
          zajmuje >30 min,"
          echo "wykonujemy czynnosc zastepcza: Code Quality."

          sudo apt-get update && sudo apt-get install -y shellcheck

```

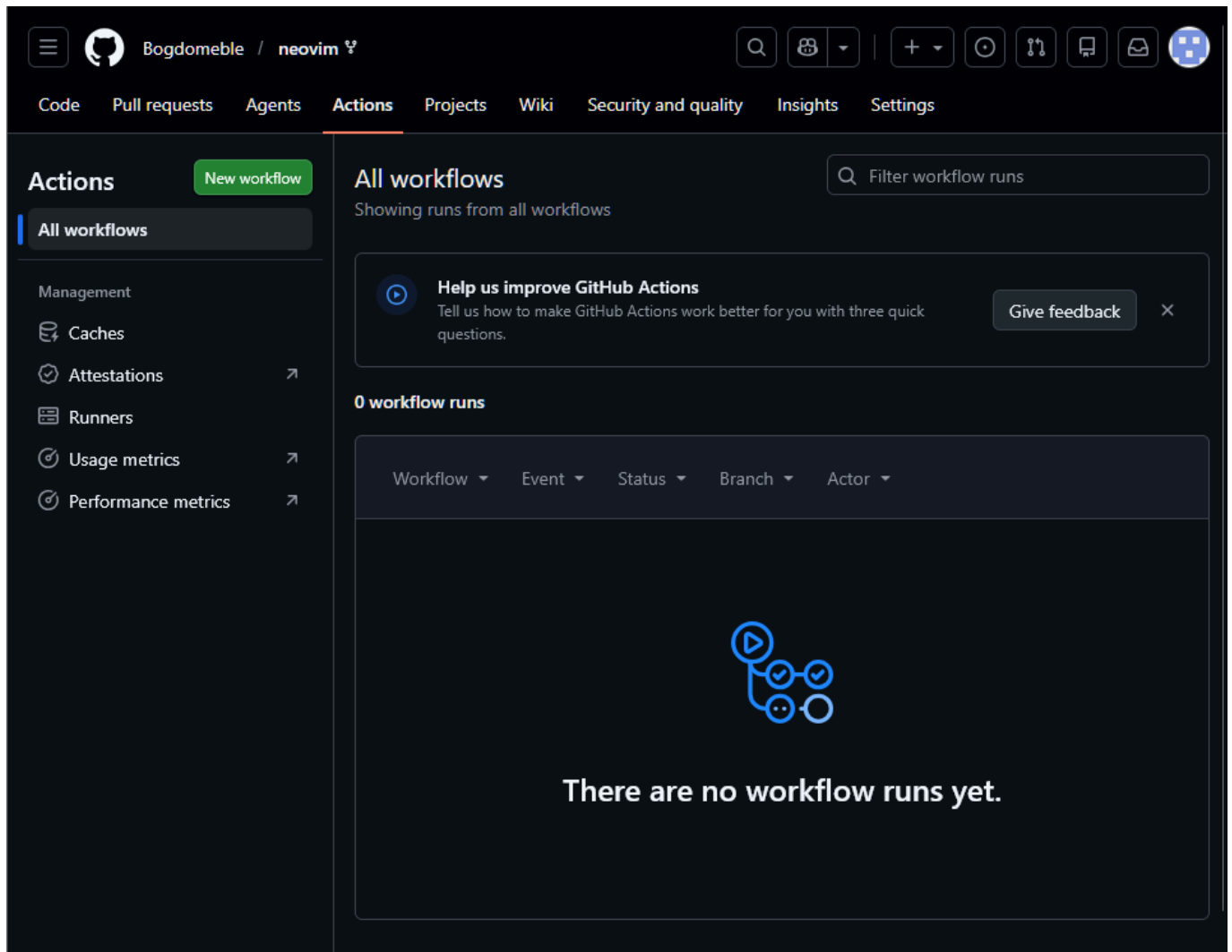
```
# Szukamy wszystkich skryptów .sh w repozytorium i sprawdzamy ich
poprawność
find . -type f -name "*.sh" -exec shellcheck {} + || true

echo "Jakość kodu powłoki sprawdzona pomyślnie!"

# Wygenerowanie pliku do artefaktu
- name: Generowanie przykładowego artefaktu
  run: |
    mkdir -p build_output
    echo "Raport Code Quality wygenerowany pomyślnie dla gałęzi
ino_dev" > build_output/raport_cq.txt
    # Pakujemy folder do archiwum .tar.gz
    tar -czvf neovim-raport.tar.gz build_output/

# Zapisanie zbudowanego artefaktu za pomocą dedykowanej akcji
- name: Udostępnienie artefaktu (Upload Artifact)
  uses: actions/upload-artifact@v4
  with:
    name: neovim-raport-artefakt
    path: neovim-raport.tar.gz
    retention-days: 5
```

Przed spushowaniem commita z plikiem konfiguracyjnym panel github actions wygląda tak:



Po spushowaniu commita pipeline jest uruchamiany i wszystko przedchodzi bez błędów:

```
● bartosz123@ansible-master:~/MD02026s_ITE/ITE/GCL1/BB419678/lab12/neovim/.github/workflows$ git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 6 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 1.12 KiB | 1.12 MiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Bogdomeble/neovim.git
  7288232660..26d789a034  ino_dev -> ino_dev
○ bartosz123@ansible-master:~/MD02026s_ITE/ITE/GCL1/BB419678/lab12/neovim/.github/workflows$
```

← Shift-left Neovim Pipeline

add shift-left.yaml #1

Re-run all jobs

...

Summary

All jobs

code-quality-and-artifact

Run details

Usage

Workflow file

Triggered via push 3 minutes ago

Status

Total duration

Artifacts

Bogdomeble pushed -o 26d789a ino_dev

Success

25s

1

shift-left.yaml

on: push

code-quality-and-artifact 22s

[] - +

Annotations

1 warning

code-quality-and-artifact

Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may...

Show more

← Shift-left Neovim Pipeline

add shift-left.yaml #1

Re-run all jobs

...

Summary

All jobs

code-quality-and-artifact

Run details

Usage

Workflow file

Annotations

1 warning

code-quality-and-artifact

succeeded 3 minutes ago in 22s

Search logs

⌂ ⚙

Set up job 1s

Checkout repository 4s

Sprawdzenie jakości kodu (Shellcheck) 13s

Generowanie przykładowego artefaktu 0s

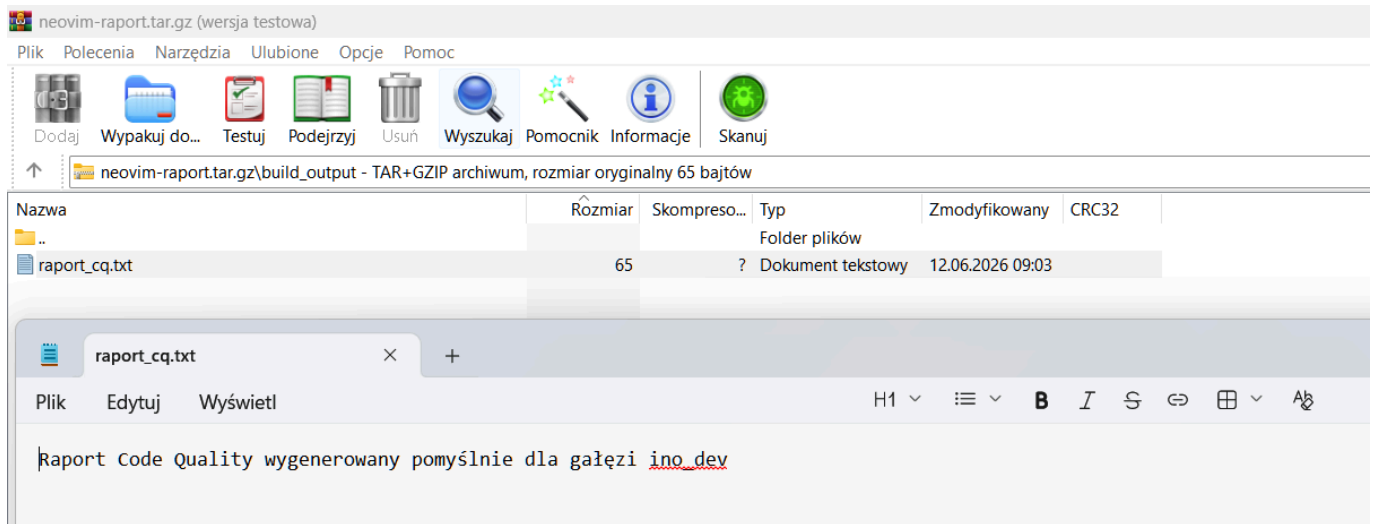
Udostępnienie artefaktu (Upload Artifact) 1s

Post Checkout repository 0s

Complete job 0s

Wynikiem pipeline'u jest właśnie raport w postaci pliku tekstowego, który można normalnie pobrać i sprawdzić na własnej maszynie:

5 / 15



Następnie dodaje kolejny plik konfiguracyjny, którego zadaniem jest przybliżenie końcowej wersji jenkinsfile. Pipeline pobiera zależności, kompiluje neovima, buduje paczkę debiana za pomocą Cpack, wykonuje mój smoke test z argumentem --headless oraz zwraca naszą paczkę .deb w tarball'u.

```
name: Full Neovim Build Pipeline

on:
  push:
    branches:
      - ino_dev

jobs:
  build-and-test:
    runs-on: ubuntu-24.04

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      # -----
      # Zbudowanie Neovima i paczki DEB (Natywnie)
      # -----
      - name: Instalacja zaleznosci systemowych
        run: |
          sudo apt-get update
          sudo apt-get install -y ninja-build gettext cmake unzip curl

      build-essential

      - name: Kompilacja Neovima ze zrodel
        run: |
          echo "Rozpoczynamy kompilacje Neovima. Moze to potrwać 10-15
          minut..."
          make CMAKE_BUILD_TYPE=Release

      - name: Pakowanie do pliku DEB (Wykorzystanie CPack)
        run: |
          cd build
```

```

cpack -G DEB

# Zmiana nazwy wygenerowanego pliku na ta, która była w Jenkinsie
mv nvim-*.deb ../neovim-final.deb
cd ..

echo "--- zależności zbudowanej paczki .deb ---"
dpkg -I ./neovim-final.deb | grep 'Depends' || true

# -----
# Deploy i Smoke Test w izolowanym kontenerze Dockera
# -----
- name: Deploy i Smoke Test (CD)
  run: |
    echo "--- Tworzymy przykładowe pliki tekstowe do smoke test'u ---"

    echo "To jest NIEZMODYFIKOWANY tekst testowy." > test_file.txt

    echo "--- Budowanie kontenera CD (Deploy) ---"
    cat << 'EOF' > Dockerfile.deploy
    FROM ubuntu:24.04
    COPY neovim-final.deb /tmp/
    COPY test_file.txt /workspace/
    WORKDIR /workspace
    ARG DEBIAN_FRONTEND=noninteractive
    RUN apt-get update && apt-get install -y /tmp/neovim-final.deb &&
rm -rf /var/lib/apt/lists/*
    EOF

    docker build -t neovim-cd-container -f Dockerfile.deploy .
    docker rm -f smoke-test-run || true

    echo "--- Wykonanie testu w kontenerze (Run & Check OUT) ---"
    docker run --name smoke-test-run neovim-cd-container nvim --
headless -c '%s/NIEZMODYFIKOWANY/ZMODYFIKOWANY/g' -c 'wq' test_file.txt

    echo "--- Weryfikacja wyniku (OUT -> OK?) ---"
    docker cp smoke-test-run:/workspace/test_file.txt
./test_file_out.txt

    if grep -q "ZMODYFIKOWANY" ./test_file_out.txt; then
        echo "Neovim poprawnie zedytował plik."
    else
        echo "Plik nie został poprawnie przetworzony!"
        exit 1
    fi

- name: Sprzątanie po Smoke Testie
  if: always() # klauzula post { always { ... } } w Jenkinsie
  run: docker rm -f smoke-test-run || true

# -----
# Archiwizacja artefaktu
# -----

```

```
- name: Archiwizacja Artefaktow (Paczka DEB)
  uses: actions/upload-artifact@v4
  with:
    name: neovim-final-deb
    path: ./neovim-final.deb
    retention-days: 5
```

Po spushowaniu commita dostaje takie wyniki:

```
● bartosz123@ansible-master:~/MDO2026s ITE/ITE/GCL1/BB419678/lab12/neovim/.github/workflows$ git add full-build.yaml
● bartosz123@ansible-master:~/MDO2026s ITE/ITE/GCL1/BB419678/lab12/neovim/.github/workflows$ git commit -m "added full build pipeline"
[ino_dev a3e235c6e1] added full build pipeline
1 file changed, 87 insertions(+)
create mode 100644 .github/workflows/full-build.yaml
● bartosz123@ansible-master:~/MDO2026s ITE/ITE/GCL1/BB419678/lab12/neovim/.github/workflows$ git push
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 6 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 1.57 KiB | 1.57 MiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Bogdomeble/neovim.git
26d789a034..a3e235c6e1 ino_dev -> ino_dev
○ bartosz123@ansible-master:~/MDO2026s ITE/ITE/GCL1/BB419678/lab12/neovim/.github/workflows$
```

The screenshot shows the GitHub Actions interface for a workflow named "Full Neovim Build Pipeline". The workflow is currently running, indicated by a yellow circle icon and the status "In progress". The workflow was triggered by a push to the "ino_dev" branch. The interface shows a summary of the workflow, including the job "build-and-test" which is currently running and has a duration of 40s. The workflow file "full-build.yaml" is also visible.

← Full Neovim Build Pipeline

added full build pipeline #1 Cancel workflow ...

Summary

All jobs

build-and-test

Run details

Usage

Workflow file

Triggered via push now

Bogdomeble pushed -> a3e235c ino_dev

Status: In progress

Total duration: —

Artifacts: —

full-build.yaml

on: push

build-and-test 40s

← Full Neovim Build Pipeline

✓ added full build pipeline #1 Re-run all jobs ...

Summary

All jobs

✓ build-and-test

Run details

Usage

Workflow file

build-and-test
succeeded now in 2m 33s

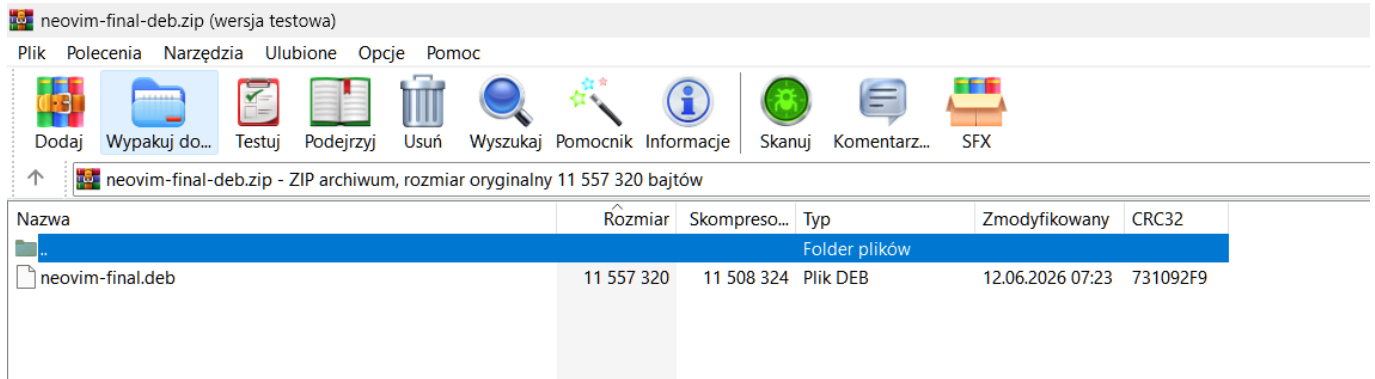
Search logs

- > ✓ Set up job 1s
- > ✓ Checkout repository 3s
- > ✓ Instalacja zaleznosci systemowych 19s
- > ✓ Kompilacja Neovima ze zrodel 1m 30s
- > ✓ Pakowanie do pliku DEB (Wykorzystanie CPack) 23s
- > ✓ Deploy i Smoke Test (CD) 14s
- > ✓ Sprzatanie po Smoke Testcie 0s
- > ✓ Archiwizacja Artefaktow (Paczka DEB) 1s
- > ✓ Post Checkout repository 0s
- > ✓ Complete job 0s

```

132 (Reading database ... 4381 files and directories currently installed.)
133 #11 6.695 Preparing to unpack /tmp/neovim-final.deb ...
134 #11 6.696 Unpacking neovim (0.13.0) ...
135 #11 7.208 Setting up neovim (0.13.0) ...
136 #11 7.230 update-alternatives: using /usr/bin/nvim to provide /usr/bin/vi (vi) in auto
mode
137 #11 7.233 update-alternatives: using /usr/bin/nvim to provide /usr/bin/vim (vim) in
auto mode
138 #11 7.235 update-alternatives: using /usr/bin/nvim to provide /usr/bin/view (view) in
auto mode
139 #11 DONE 7.3s
140
141 #12 exporting to image
142 #12 exporting layers
143 #12 exporting layers 1.2s done
144 #12 writing image
sha256:23407e8b54f22b0f31eeea0cd3bfd4ab867f6739d136b77d848d493289a36d26 done
145 #12 naming to docker.io/library/neovim-cd-container done
146 #12 DONE 1.2s
147 Error response from daemon: No such container: smoke-test-run
148 --- Wykonanie testu w kontenerze (Run & Check OUT) ---
149 --- Weryfikacja wyniku (OUT -> OK?) ---
150 Neovim poprawnie zedytował plik.
151 "test_file.txt" "test_file.txt" 1L, 37B written

```



Jak widać, wszystkie kroki przeszły poprawnie i bezbłędnie. Jeżeli dodałbym do tego pipeline'u testy jednostkowe dla całego oprogramowania, zajęłoby to niestety ponad 30 min, dlatego proces ten został o ten etap "uszczuplony".

Wnioski

Temat 13

Cel zajęć: Wdrażanie na zarządzalne kontenery w chmurze.

Przed zajęciami miałem już zarejestrowane konto uczelniane na platformie Azure, dlatego nie muszę teraz zajmować się procesem rejestracji.

Na początku łączę się z azure cloud shell, aby utworzyć grupę zasobów potrzebnych w tym ćwiczeniu. Wynikiem tego polecenia jest obiekt JSON, który zwraca konfigurację naszej grupy (region, tagi, nazwa itd.).

```
bartosz [ ~ ]$ az group create --name Lab13-ResourceGroup --location westeurope
{
  "id": "/subscriptions/cb842ff3-17a8-404b-8439-6a9d8336f6da/resourceGroups/Lab13-ResourceGroup",
  "location": "westeurope",
  "managedBy": null,
  "name": "Lab13-ResourceGroup",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
bartosz [ ~ ]$
```

Następnie publikuje kontener mojej aplikacji nginx w wersji v3 na dockerHub w celu użycia jego w konterzene Azure:

```

● bartosz123@ansible-master:~/MD02026s_ITE/ITE/GCL1/BB419678/lab11$ docker tag bogdomeble/moj-app:v3 bartbod/moj-app:v3
● bartosz123@ansible-master:~/MD02026s_ITE/ITE/GCL1/BB419678/lab11$ docker push bartbod/moj-app:v3
The push refers to repository [docker.io/bartbod/moj-app]
e314c84304d8: Pushed
5519239b187c: Pushed
9a2cbc774b1f: Pushed
ad8253b95bde: Pushed
906af8ee0f8a: Pushed
2d9b3b001567: Pushed
2b05bd5bca95: Pushed
b910427bd94b: Pushed
29df493baa13: Pushed
v3: digest: sha256:558a8e4aa39a646a0d6716c1767d972a14f067e62e007272f9f62242c1da3ab3 size: 2196
○ bartosz123@ansible-master:~/MD02026s_ITE/ITE/GCL1/BB419678/lab11$

```

Muszę trochę poczekać, gdyż niestety mamy rate limiting od strony DockerHub'a dla usług Azure.

```

bartosz [ ~ ]$ az provider show -n Microsoft.ContainerInstance --query "registrationState"
"Registering"
bartosz [ ~ ]$ az provider show -n Microsoft.ContainerInstance --query "registrationState"
"Registering"
bartosz [ ~ ]$ az provider show -n Microsoft.ContainerInstance --query "registrationState"
"Registering"
bartosz [ ~ ]$ az provider show -n Microsoft.ContainerInstance --query "registrationState"
"Registering"
bartosz [ ~ ]$ az provider show -n Microsoft.ContainerInstance --query "registrationState"
"Registering"

```

Po około 20 minutach mogę już stworzyć mój kontener, który działa na porcie 80 z obrazem pobranym z mojego DockerHuba o nazwie `moj-app-bartbod123`, który używa 1 vCPU i 1 GB RAM w systemie Linux (Domyślam się, że jest to apline Linux).

```

bartosz [ ~ ]$ az container create \
  --resource-group Lab13-ResourceGroup \
  --name moja-aplikacja \
  --image bartbod/moj-app:v3 \
  --dns-name-label moj-app-bartbod-123 \
  --ports 80 \
  --cpu 1 \
  --memory 1 \
  --os-type Linux

```

Po utworzeniu kontenera dostaje odpowiedź w postaci obiektu JSON, który daje mi dużą ilość informacji na temat stworzonego kontenera:

```

{
  "confidentialComputeProperties": null,
  "containerGroupProfile": null,
  "containers": [
    {
      "command": null,
      "configMap": {
        "keyValuePairs": {}
      },
    },
  ],
}

```

```

"environmentVariables": [],
"image": "bartbod/moj-app:v3",
"instanceView": {
  "currentState": {
    "detailStatus": "",
    "exitCode": null,
    "finishTime": null,
    "startTime": "2026-06-21T17:31:45.935000+00:00",
    "state": "Running"
  },
  "events": [
    {
      "count": 1,
      "firstTimestamp": "2026-06-21T17:31:45+00:00",
      "lastTimestamp": "2026-06-21T17:31:45+00:00",
      "message": "Started container",
      "name": "Started",
      "type": "Normal"
    },
    {
      "count": 1,
      "firstTimestamp": "2026-06-21T17:31:35+00:00",
      "lastTimestamp": "2026-06-21T17:31:35+00:00",
      "message": "Successfully pulled image \"bartbod/moj-
app@sha256:558a8e4aa39a646a0d6716c1767d972a14f067e62e007272f9f62242c1da3ab3
\"",
      "name": "Pulled",
      "type": "Normal"
    },
    {
      "count": 1,
      "firstTimestamp": "2026-06-21T17:31:25+00:00",
      "lastTimestamp": "2026-06-21T17:31:25+00:00",
      "message": "pulling image \"bartbod/moj-
app@sha256:558a8e4aa39a646a0d6716c1767d972a14f067e62e007272f9f62242c1da3ab3
\"",
      "name": "Pulling",
      "type": "Normal"
    }
  ],
  "previousState": null,
  "restartCount": 0
},
"livenessProbe": null,
"name": "moja-aplikacja",
"ports": [
  {
    "port": 80,
    "protocol": "TCP"
  }
],
"readinessProbe": null,
"resources": {
  "limits": null,

```

```
    "requests": {
      "cpu": 1.0,
      "gpu": null,
      "memoryInGb": 1.0
    }
  },
  "securityContext": null,
  "volumeMounts": null
}
],
"diagnostics": null,
"dnsConfig": null,
"encryptionProperties": null,
"extensions": null,
"id": "/subscriptions/cb842ff3-17a8-404b-8439-6a9d8336f6da/resourceGroups/Lab13-ResourceGroup/providers/Microsoft.ContainerInstance/containerGroups/moja-aplikacja",
"identity": null,
"imageRegistryCredentials": null,
"initContainers": [],
"instanceView": {
  "events": [],
  "state": "Running"
},
"ipAddress": {
  "autoGeneratedDomainNameLabelScope": "Unsecure",
  "dnsNameLabel": "moj-app-bartbod-123",
  "fqdn": "moj-app-bartbod-123.westeurope.azurecontainer.io",
  "ip": "20.54.201.53",
  "ports": [
    {
      "port": 80,
      "protocol": "TCP"
    }
  ]
},
"type": "Public"
},
"isCreatedFromStandbyPool": false,
"location": "westeurope",
"name": "moja-aplikacja",
"osType": "Linux",
"priority": null,
"provisioningState": "Succeeded",
"resourceGroup": "Lab13-ResourceGroup",
"restartPolicy": null,
"sku": "Standard",
"standbyPoolProfile": null,
"subnetIds": null,
"tags": {},
"type": "Microsoft.ContainerInstance/containerGroups",
"volumes": null,
```

```
"zones": null
}
```

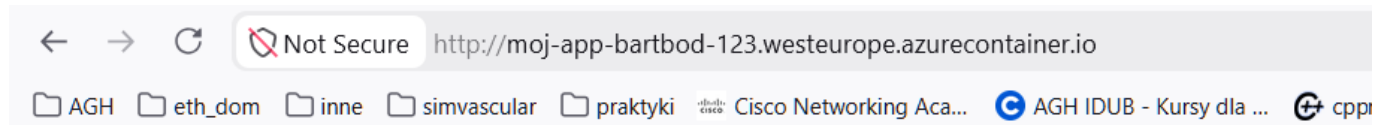
Dodatkowo sprawdzam istnienie kontenera za pomocą powłoki - wynikiem jest adres, który mogę wpisać do przeglądarki i sprawdzić działanie mojej aplikacji statycznej.

```
bartosz [ ~ ]$ az container show \
  --resource-group Lab13-ResourceGroup \
  --name moja-aplikacja \
  --query "{FQDN:ipAddress.fqdn, Stan:provisioningState}" \
  --out table
```

FQDN	Stan
moj-app-bartbod-123.westeurope.azurecontainer.io	Succeeded

```
bartosz [ ~ ]$
```

Jak widać, wszystko działa zgodnie z naszą konfiguracją. Przez to, że nasze połączenie nie jest skonfigurowane pod HTTPS, dostajemy informacje "Not Secure" od firefoxa'a. Na potrzeby tego laboratorium taka konfiguracja jest wystarczająca.



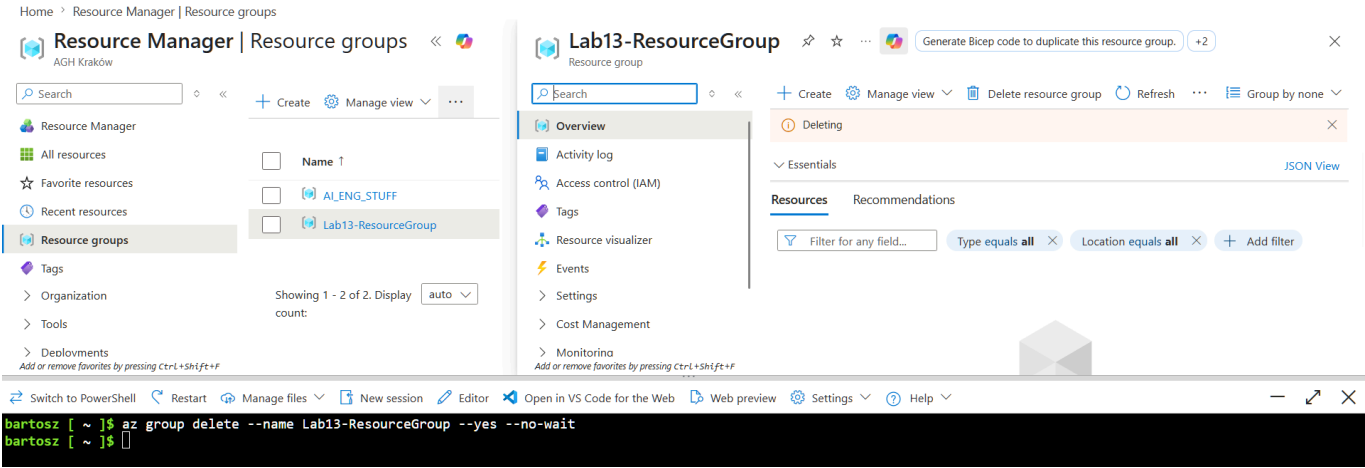
Witaj z chmury Azure! Aplikacja w wersji V3

Następnie sprawdzam logi maszyny, w której widzimy logi naszej aplikacji oraz żądania HTTP jak i błędy not found 404 na niezdefiniowanych ścieżkach api.

```
bartosz [ ~ ]$ az container logs \
  --resource-group Lab13-ResourceGroup \
  --name moja-aplikacja
```

```
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/06/21 17:31:46 [notice] 19#19: using the "epoll" event method
2026/06/21 17:31:46 [notice] 19#19: nginx/1.31.1
2026/06/21 17:31:46 [notice] 19#19: built by gcc 15.2.0 (Alpine 15.2.0)
2026/06/21 17:31:46 [notice] 19#19: OS: Linux 6.1.146.1-microsoft-standard
2026/06/21 17:31:46 [notice] 19#19: getrlimit(RLIMIT_NOFILE): 1024:1048576
2026/06/21 17:31:46 [notice] 19#19: start worker processes
2026/06/21 17:31:46 [notice] 19#19: start worker process 46
10.92.0.9 - - [21/Jun/2026:17:33:13 +0000] "GET / HTTP/1.0" 200 53 "-" "Shodan-Pull/1.0" "-"
10.92.0.11 - - [21/Jun/2026:17:33:49 +0000] "GET / HTTP/1.1" 200 53 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:152.0) Gecko/20100101 Firefox/152.0" "-"
10.92.0.9 - - [21/Jun/2026:17:33:49 +0000] "GET / HTTP/1.1" 200 53 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:152.0) Gecko/20100101 Firefox/152.0" "-"
10.92.0.9 - - [21/Jun/2026:17:33:49 +0000] "GET /favicon.ico HTTP/1.1" 404 153 "http://moj-app-bartbod-123.westeurope.azurecontainer.io/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:152.0) Gecko/20100101 Firefox/152.0" "-"
2026/06/21 17:33:49 [error] 46#46: *19 open() "/usr/share/nginx/html/favicon.ico" failed (2: No such file or directory), client: 10.92.0.9, server: localhost, request: "GET /favicon.ico HTTP/1.1", host: "moj-app-bartbod-123.westeurope.azurecontainer.io", referer: "http://moj-app-bartbod-123.westeurope.azurecontainer.io/"
```

Na koniec usuwam poleceniem `az group delete --name --yes --no-wait` moją grupę zasobów, aby nie marnować niepotrzebnie moich przydzielonych kredytów.



Wnioski